APPLICATION HEALTH

Monitoring, Performance & Incident Management

OBJECTIVE

To provide a comprehensive understanding of Application Health Monitoring, covering real-time performance metrics, runtime health indicators, alerting workflows, cross-application tracing, and key tooling. Enabling teams to detect, diagnose, and resolve incidents faster, reduce downtime, and maintain optimal user experience.

Detect Early

Identify performance degradations before they become user-impacting incidents.

Alert Smartly

Design alerting rules that minimize noise and surface actionable signals.

Trace Across Services

Correlate requests across microservices to pinpoint root cause quickly.

Leverage Tooling

Use Azure Monitor, Dynatrace & Grafana to automate and accelerate response.

AGENDA

01 **Application Health Overview**
Core concepts, signals & dimensions

02 **Real-Time Performance Monitoring**
Latency, throughput, error rates & metrics

03 **Application Health Indicators**
Availability checks & runtime health

04 **How to Monitor**
Response time, end-user experience, transactions & tracing

05 **Health Monitoring Deep Dive**
Port/URL checks, GC monitoring

06 **Alerting & Incident Workflows**
Alert patterns that drive incident response

07 **Monitoring Tools**
Azure Monitor, Dynatrace, Grafana

08 **Key Takeaways**
Summary of critical insights

APPLICATION HEALTH OVERVIEW

Application Health Monitoring focuses on detecting degradations or failures before they become major incidents. It combines performance telemetry, availability checks, and runtime health indicators to determine whether an application is functioning correctly.

01

Real-Time Application Performance Monitoring

Tracks quantitative metrics like latency, throughput, error rates, and resource utilization under real workload conditions.

02

Runtime Health Monitoring

Monitors availability, runtime state, infrastructure health, dependency status, and resource exhaustion signals.

Telemetry Signals → Alerting Systems → Incident Management → Remediation

APPLICATION PERFORMANCE METRICS

COMMON METRICS

- Latency (response time)
- Throughput (requests/sec)
- Error Rates
- Transaction Durations
- Resource Utilization
- Queue Lengths
- Dependency Latency (DB, APIs)

METRIC CHANGE → POSSIBLE INCIDENT

Metric Change	Possible Incident
Increased response time	DB slowdown, network latency
Spike in error rate	API failure or code defect
Reduced throughput	Thread pool exhaustion
Transaction duration spike	Downstream service degradation

HOW PERFORMANCE METRICS FEED ALERTING WORKFLOWS

Threshold Alert

IF p95 response time > 1s for 5 min THEN trigger incident

Error Rate Alert

IF error rate > 5% over 2 min THEN page on-call engineer

Rate-of-Change

IF latency increases > 200% within 3 min THEN raise high priority alert

SLO-Based Alert

IF error budget burn rate > threshold THEN trigger SRE incident

ALERTS DRIVE:

- PagerDuty / Opsgenie Pages
- Incident Ticket Creation
- Slack / Teams Alerts
- Automated Remediation

APPLICATION HEALTH INDICATORS

Healthy | Degraded | Unhealthy

Health Indicator	What It Detects
HTTP health endpoints	Application process alive
Port checks	Service reachable
Synthetic transactions	Functional availability
Dependency checks	DB / API health
Runtime metrics	Memory, GC, threads

Situation	Health Signal
Application crash	Health endpoint unreachable
DB connection exhaustion	Health degraded
Memory leak	Health gradually degrading
External API outage	Partial service health

KUBERNETES HEALTH ENDPOINTS

/health

Overall operational status

/ready

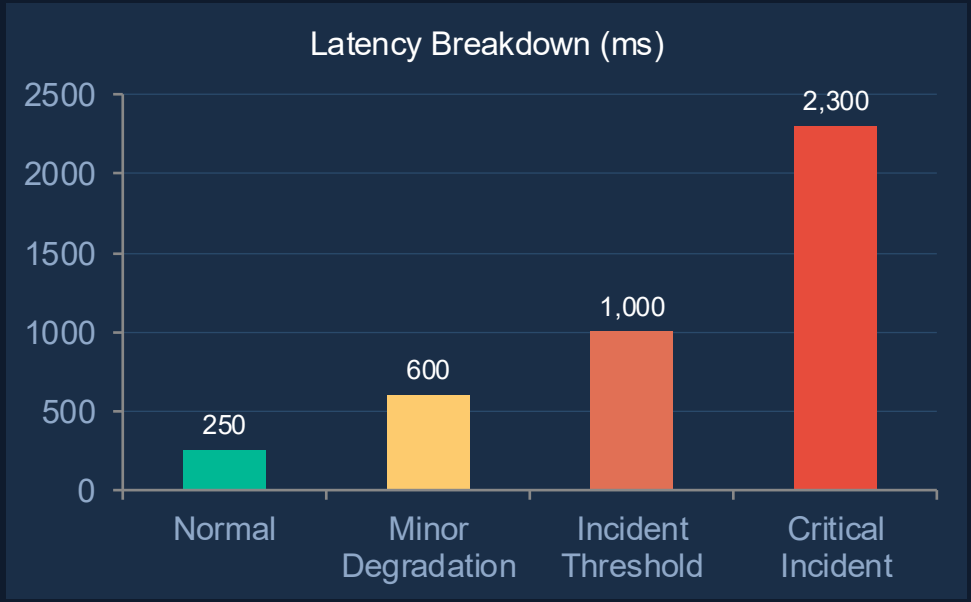
Can it accept traffic?

/liveness

Is the process alive?

HOW TO MONITOR: RESPONSE TIME & END USER EXPERIENCE

Total Response Time = App Processing + Database Time + External API Calls + Network Latency



END USER EXPERIENCE METRICS

Largest Contentful Paint (LCP)
< 2.5s

First Input Delay (FID)
< 100ms

Page Load Time
< 3s

Alert: IF page_load_time > 4s for 10% of users THEN create performance incident

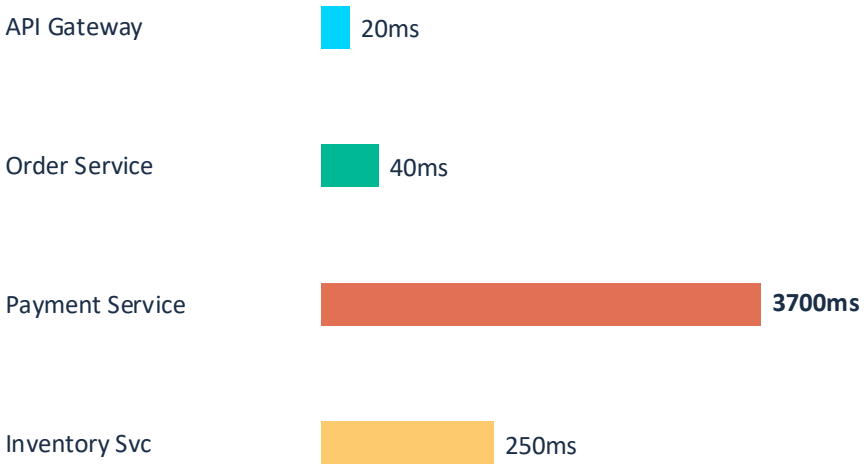
TRANSACTIONS & CROSS-APPLICATION TRACING

Transaction	Normal Duration	Incident Value
Login Flow	200ms	—
Checkout Workflow	800ms	8 seconds ⚠
Report Generation	1.2s	—
Search Query	150ms	—

MICROSERVICE REQUEST PATH

API Gateway → Auth Service → Order Service → Inventory Service → Payment Gateway

DISTRIBUTED TRACE: CHECKOUT REQUEST



⚠ **Payment Service is the root cause: 3.7s of 4s total latency**

HEALTH MONITORING: PORT/URL & GARBAGE COLLECTION

PORT / URL MONITORING

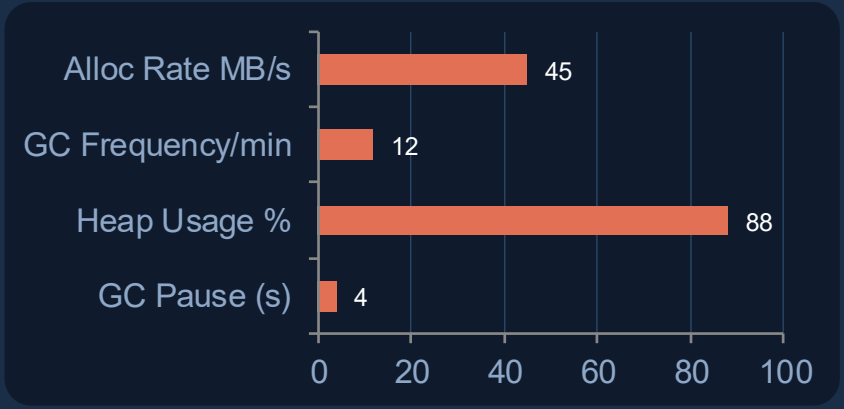
Check Type	Example
TCP Port Check	Port 443 open
HTTP Response	Status 200
Response Time	< 2 seconds
Content Check	Response contains 'OK'

```
GET https://api.example.com/health
```

Alert: IF health_endpoint_failure > 3 checks THEN page on-call immediately

Detects: App crash | DNS issues | Load balancer misconfiguration

GARBAGE COLLECTION (GC) MONITORING



Alert: IF GC pause > 2s AND heap > 85% THEN trigger memory incident

Applies to: Java JVM | .NET CLR | Go runtime

Excessive GC causes latency spikes, app freezes, memory leaks & crashes

SAMPLE MONITORING TOOLS

Azure Monitor

Microsoft Cloud-Native Observability

- Distributed tracing & dependency tracking
- Request latency & failure rate tracking
- Availability tests & endpoint checks
- Dynamic thresholds & action groups
- Application Insights APM integration

Dynatrace

AI-Driven Observability Platform

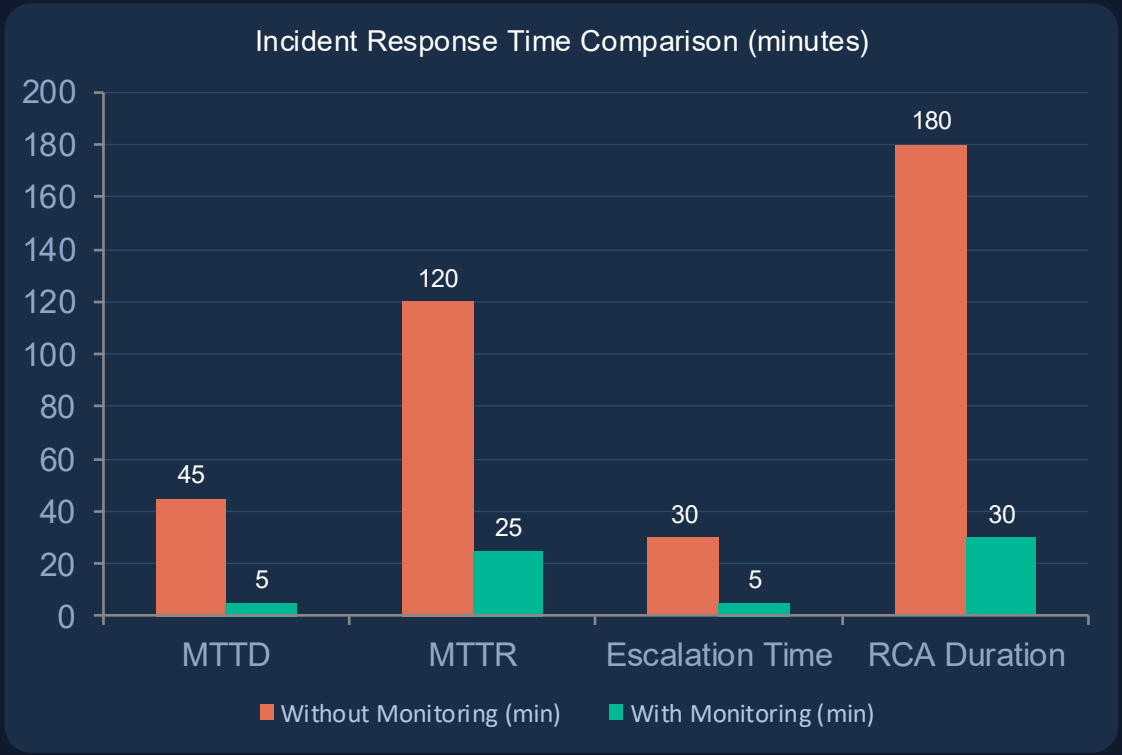
- Auto instrumentation & Smartscape topology
- PurePath distributed tracing
- Automated health scores per service
- Davis AI: root-cause correlation
- Problem detection with alert storm suppression

Grafana

Open Observability & Visualization

- Integrates: Prometheus, Loki, Elasticsearch
- Dashboards: latency, error rate, throughput
- SRE war room visualization
- Alerting with multi-source correlation
- p95/p99 latency panels by service

IMPACT OF APPLICATION HEALTH MONITORING



9x Faster MTTD

5x Faster MTTR

6x Less RCA Time

KEY TAKEAWAYS

01

Proactive Detection Over Reactive Response

Performance metrics (latency, throughput, error rates) are the earliest indicators of emerging incidents, enabling detection before full service failure.

02

Two Dimensions: Performance + Health

Real-time performance monitoring and runtime health monitoring work together to answer both 'How fast?' and 'Is it up and healthy?'

03

Percentiles > Averages

p95 and p99 latency metrics expose the worst-case experience for real users. Averages mask performance spikes that drive user dissatisfaction.

04

Distributed Tracing Pinpoints Root Cause

In microservice environments, cross-application tracing reduces RCA from hours to minutes by showing exactly where latency originates.

05

Alert Quality Reduces Noise

Well-designed alert rules (SLO-based, rate-of-change, dynamic thresholds) reduce alert fatigue and ensure on-call engineers are paged for real incidents.

06

Tooling: Automate & Correlate

Azure Monitor, Dynatrace, and Grafana provide automation, AI-driven root cause analysis, and rich visualization, dramatically reducing MTTD and MTTR.